Wilhelm Büchner University of Applied Sciences

Projektseminar Master

Coasterbot

Softwaredokumentation



Dennis Borutta, B. Eng dennis.borutta@outlook.com
Stefan Etringer, B. Eng mail@stefan.works
Gereon Such, M. Sc. gereonsuch@gmail.com

Versionshistorie

Version	Datum	Bearbeiter	Änderungen
1	08.09.2026	Etringer	Erstfassung der Softwaredokumentation
2	08.09.2026	Etringer	Lokalisierung um Prinzip und Grenzen ergänzt
3	13.09.2026	Etringer	Offene Punkte aus Hardware-Rückmeldung zu `RobotHAL` ergänzt
4	13.09.2026	Etringer	Kantensensoren liefern jetzt `bool` statt `float`, Kalibrierung in `WebotsHAL` gekapselt
5	13.09.2026	Etringer	`enum WheelId` auf `WHEEL_LEFT`/`WHEEL_RIGHT` reduziert (ein Encoder je Seite)
6	14.09.2026	Etringer	Geglättete Kartenanpassung und digitales Kantensignal umgesetzt; Ausblick Tischerkundung ergänzt
7	14.09.2026	Etringer	Ausblick Tischerkundung zu Stufenplan (Stufe 0 bis 7) ausgearbeitet
8	16.09.2026	Etringer	Gyro-Schnittstelle um Hardware-Hinweise ergänzt (Kalibrierung, Skid-Steer)
9	20.09.2026	Etringer	Inertialsensor (GY-521/MPU6050) in `WebotsHAL::getGyroZ()` simuliert (Abschnitt 4.3); `randomSeed` fest gesetzt
10	21.09.2026	Etringer	Open-/Closed-Loop-Motorregelung implementiert und verglichen (`MODE_MOTOR_CONTROL_TEST`, Abschnitt 4.4)
11	28.09.2026	Such	Besonderheiten der Hardware Implementierung hinzugefügt
12	28.09.2026	Ertringer, Borutta	Statemachine ergänzt
13	19.10.2026	Etringer	Codeblock in „Non-blocking Implementierung“ angepasst. Kapitel 16 aktualisiert (realer Encoder, Untersetzerhandling in `coasterbot_V_01_00`)

Inhaltsverzeichnis

1 Einleitung **5**
- 1.1 Zweck des Dokuments . . . 5
- 1.2 Gegenstand und Abgrenzung . . . 5
- 1.3 Begriffe und Abkürzungen . . . 6

2 Softwarearchitektur **6**
- 2.1 Architekturprinzip . . . 6
- 2.2 Abbildung auf die Pflichtenheft-Architektur . . . 7
- 2.3 Datenfluss im kombinierten Betrieb . . . 8

3 Laufzeitumgebung und Projektstruktur **9**
- 3.1 Werkzeuge . . . 9
- 3.2 Quelldateien . . . 9

4 Hardwareabstraktion **10**
- 4.1 Interface `RobotHAL` . . . 10
- 4.2 Implementierung `WebotsHAL` . . . 11
- 4.3 Gyro-Simulation: GY-521/MPU-6050 . . . 12
- 4.4 Motorregelung: Open-Loop vs. Closed-Loop im Vergleich . . . 13

5 Lokalisierungskomponente **15**
- 5.1 Prinzip . . . 15
- 5.2 Verfahren . . . 16
- 5.3 Schnittstelle . . . 17
- 5.4 Grenzen der Lokalisierung . . . 17

6 Umgebungsmodell **17**
- 6.1 Aufbau . . . 17
- 6.2 Funktionen . . . 18
- 6.3 Geglättete Live-Hindernismeldung . . . 18

7 Globale Pfadplanung mit A* **19**
- 7.1 Algorithmus . . . 19
- 7.2 Auswahlbegründung . . . 20

8 Dynamische Neuplanung mit D* Lite **20**
- 8.1 Verfahren . . . 20

9 Lokale Kollisionsvermeidung und Bewegungssteuerung mit DWA **21**
- 9.1 Dynamic Window Approach . . . 21
- 9.2 Pfadverfolgung (Pure Pursuit) . . . 21
- 9.3 Hindernisse . . . 21
- 9.4 Bewegungssteuerung . . . 22

10 Sicherheitskomponente **22**
10.1 Funktion . . . 22
10.2 Schnittstelle . . . 23
10.3 Fehlerüberwachung (`FaultMonitor`) . . . 23
10.4 Ablauf `MODE_FAULT_INJECTION` . . . 24

11 Reaktive Verhaltenslogik **25**

12 Ablaufsteuerung und Betriebsarten **25**
12.1 Ablauf `MODE_SIM_TEST` . . . 26
12.2 Vorschlag: Missionsablaufsteuerung . . . 26

13 Simulationsmodell **28**
13.1 Robotermodell (PROTO) . . . 28
13.2 Welten . . . 28
13.3 Reproduzierbarkeit . . . 29

14 Systemüberwachung **29**

15 Abbildung auf Anforderungen und Testfälle **30**
15.1 Status der Simulationstestfälle . . . 31

16 Grenzen und offene Punkte **32**

17 Implementierung auf Hardware **32**
17.1 Non-blocking Implementierung . . . 33
17.2 Inertialsystem Normierung . . . 34

18 Anhang A: Schnittstelle `RobotHAL` **35**

19 Anhang B: Wesentliche Parameter **35**

1 Einleitung

1.1 Zweck des Dokuments

Dieses Dokument beschreibt die **Software der Simulation des Coasterbots** sowie deren Funktionen. Dokumentiert wird sowohl die Softwarearchitektur, als auch die einzelnen Softwarekomponenten, ihre Schnittstellen und das Laufzeitverhalten in der Simulationsumgebung.

Grundlage der Dokumentation sind die im Projekt erstellten Anforderungs- und Testdokumente:

- **Lastenheft**: funktionale und nichtfunktionale Anforderungen des Auftraggebers (Kennungen `NAV-*`, `SAF-*`, `MON-*`, `NFA-*`, `SIM-*`).
- **Pflichtenheft**: die geplante technische Umsetzung, die geschichtete System- und Softwarearchitektur (Hardwaretreiber, Hardwareabstraktion, Systemdienste, Funktionskomponenten, Anwendungsebene, Simulationsebene).
- **Testkonzept**: Teststrategie und Simulationstestfälle (`ST-SIM-001` bis `ST-SIM-008`).
- **Alternativenbeschreibung**: Auswahl der Navigations- und Lokalisierungsalgorithmen (A*, D* Lite, DWA, Sensorfusion).
- **Stückliste (BOM)**: die für die reale Hardware festgelegten Bauteile, u.a. der Inertialsensor (Position 11: „ICQUANZX GY-521 MPU6050“), den die Simulation in Abschnitt 4.3 nachbildet.

Die Software wurde anhand der im Pflichtenheft beschriebenen Abstraktion umgesetzt: Die Systemlogik ist vollständig von der Hardware entkoppelt und kommuniziert über eine Hardwareabstraktionsschicht. Dadurch sind die Funktionen im Quellcode sowohl in der Simulation (Webots) als auch auf der Zielhardware (Mikrocontroller) lauffähig.

1.2 Gegenstand und Abgrenzung

Gegenstand dieser Dokumentation ist die Software des Webots-Controllers `coasterbot` (C++), die den simulierten Roboter steuert. Dokumentiert werden:

- die Hardwareabstraktion (`RobotHAL` / `WebotsHAL`),
- die Lokalisierung (`PoseEstimator`),
- das Umgebungsmodell (`OccupancyGrid`),
- die globale Pfadplanung (`astarPlan`, A*),
- die dynamische Neuplanung (`DStarLite`, D* Lite),
- die lokale Kollisionsvermeidung und Bewegungssteuerung (`DwaPlanner`),
- die Sicherheitskomponente (`SafetyMonitor`),
- die Fehlerüberwachung (`FaultMonitor`),
- die reaktive Verhaltenslogik (`RobotLogic`),

- die Ablaufsteuerung und Betriebsarten (`main.cpp`),
- das Simulationsmodell (Webots-Welten und `Coasterbot`-PROTO).

Nicht Bestandteil dieser Dokumentation bzw. der aktuellen Umsetzung sind:

- das Untersetzerhandling (Aufnahme/Ablage von Getränkeuntersetzern, `CST-*`) inkl. der Manipulationsplanung (IK + RRT*),
- die missionsbezogene Ablaufsteuerung `IDLE` → `SEARCH_COASTER` → `PICK_UP` → `TRANSPORT` → `PLACE` → `VERIFY` → `DONE`,
- eine probabilistische Sensorfusion (EKF/UKF), es wird ein Komplementärfilter verwendet (siehe Abschnitt 5).

Der Umsetzungsstand bezieht sich auf den **Maturity Level 1** (Prototyp): autonome Navigation, Lokalisierung, sichere Bewegung und Tischkantenerkennung.

1.3 Begriffe und Abkürzungen

Begriff	**Bedeutung**
HAL	Hardware Abstraction Layer, Hardwareabstraktionsschicht
PROTO	Webots-Modellbeschreibung eines Roboters oder Objekts
Pose	Lage des Roboters in der Ebene: (x, y, θ)
Belegungskarte	Rasterkarte der Fläche, Zellen frei oder belegt (occupancy grid)
A*	heuristischer Graphsuch-Algorithmus für die globale Pfadplanung
D* Lite	inkrementeller Neuplaner (Koenig & Likhachev, 2002)
DWA	Dynamic Window Approach, lokale Bewegungsplanung im Geschwindigkeitsraum
Carrot-Punkt	vorausliegender Zielpunkt auf dem Pfad (Pure Pursuit)
Skid-Steer	Panzerlenkung: Lenkung über Drehzahldifferenz linker/rechter Räder
Regelzyklus	ein Simulationsschritt (`basicTimeStep`), typ. 32 ms

2 Softwarearchitektur

2.1 Architekturprinzip

Die Software folgt dem im Pflichtenheft (Kapitel „Softwarekonzept“) beschriebenen **geschichteten, komponentenbasierten** Architekturmodell. Jede Schicht hat eine abgegrenzte Verantwortlichkeit und kommuniziert über Schnittstellen mit den benachbarten Schichten. Höhere Schichten kennen keine Hardware- oder Simulationsdetails.

Die zentrale Abstraktion ist die Hardwareabstraktionsschicht `RobotHAL`. Diese ist ein abstraktes C++ Interface, das lose gekoppelt *weder* Webots *noch* eine konkrete Mikrocontroller-Bibliothek kennt. Alle Funktionskomponenten (Lokalisierung, Planung, Sicherheit) sind gegen dieses Interface programmiert. Dadurch werden die Anforderungen `NFA-ARC-005`, `NFA-MAI-001/002` und `NFA-SIM-004` erfüllt.

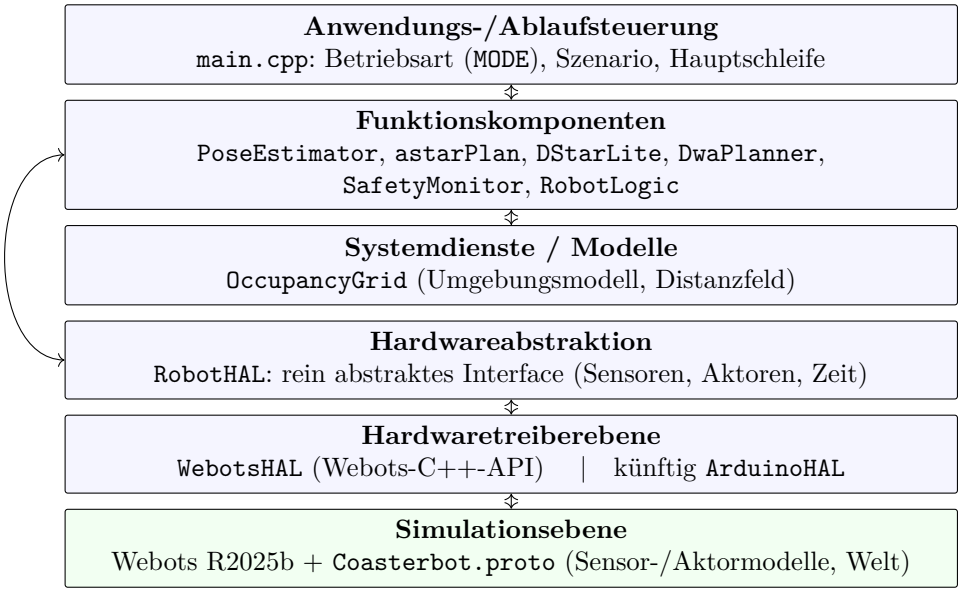


Abbildung 1: Schichtenmodell der Simulationssoftware. Die Funktionskomponenten greifen über `RobotHAL` auf Sensoren und Aktoren zu.

2.2 Abbildung auf die Pflichtenheft-Architektur

Tabelle 3 ordnet die Architekturebenen und Funktionskomponenten aus dem Pflichtenheft den Quelldateien des Simulationscontrollers zu.

Tabelle 3: Zuordnung Pflichtenheft-Architektur → Umsetzung.

Pflichtenheft	**Umsetzung (Datei / Klasse)**	**Anforderung**
Hardwaretreiberebene	`webots_hal.cpp` (Geräte-Handles)	keine
Hardwareabstraktionsebene	`robot_hal.h` (`RobotHAL`), `webots_hal` (`WebotsHAL`)	NFA-ARC-005
Systemdienste / Umgebungsmodell	`occupancy_grid` (`OccupancyGrid`)	SIM-004
Navigationskomponente	`astar` (A*), `dstar_lite` (D* Lite)	NAV-005/008/009
Lokalisierungskomponente	`pose_estimator` (`PoseEstimator`)	NAV-006, NAV-007

Pflichtenheft	Umsetzung (Datei / Klasse)	Anforderung
Bewegungssteuerung	`dwa_planner` (`DwaPlanner::driveVW`), `robot_hal.h`	HW-ACT-002/003, SAF-004
Sicherheitskomponente / Sicherheitslogik	`safety_monitor` (`SafetyMonitor`)	NAV-003/004, SAF-001..005
Systemüberwachung	Konsolenprotokoll je Betriebsart in `main.cpp`	MON-001..005
Anwendungsebene	`main.cpp` (`MODE`, Szenarien, Hauptschleifen)	keine
Simulationsebene	`worlds/*.wbt`, `Coasterbot.proto`	NFA-SIM-001..005
Reaktive Verhaltenslogik (Bring-up)	`robot_logic` (`RobotLogic`)	NAV-001/002, SAF-003

2.3 Datenfluss im kombinierten Betrieb

Abbildung 2 zeigt den Datenfluss der Betriebsart `MODE_SIM_TEST`, in der die Komponenten zusammenwirken (Grundlage der Testfälle `ST-SIM-001` bis `ST-SIM-004`).

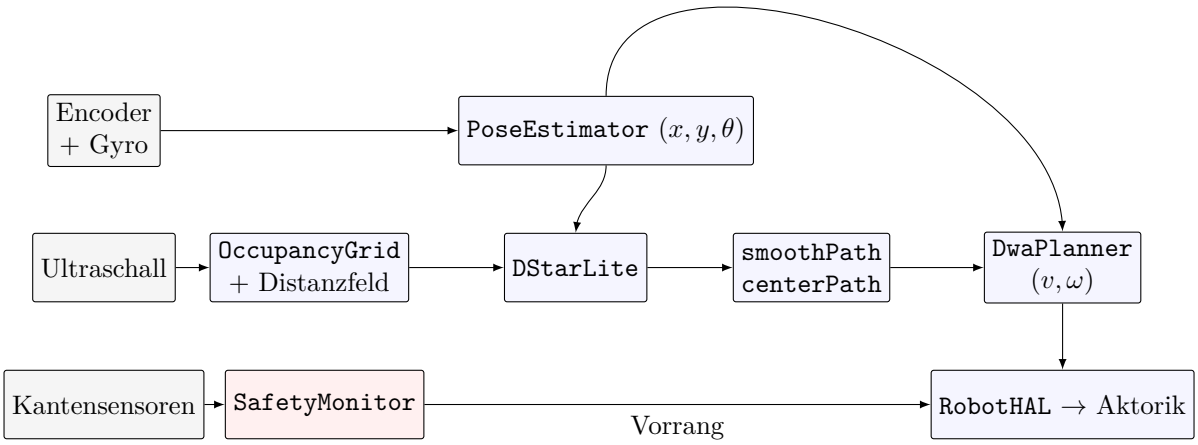


Abbildung 2: Datenfluss in `MODE_SIM_TEST`. Die Sicherheitskomponente wird in jedem Regelzyklus *vor* der Navigation ausgewertet und übernimmt bei einem Kantenereignis die Aktorik.

Ablauf eines Regelzyklus:

1. `hal.step()`: Simulationsschritt, neue Sensorwerte.

2. `pose.update()`: Pose aus Radwinkeln und Gyro fortschreiben.
3. `safety.update()`: Kantensensoren prüfen. Beinhaltet die Sicherheitskomponente der Aktorik, letzte Funktion im Regelzyklus.
4. Kartenpflege: Ein per Sensor erkanntes, unkartiertes Hindernis bzw. eine erkannte Kante wird in die Belegungskarte eingetragen und `der DStarLite` Algorithmus erneuert.
5. `planner.setStart()` / `computeShortestPath()`: Pfad inkrementell aktualisieren.
6. `smoothPath()` + `centerPath()`: Pfad glätten und die Korridor erneuern.
7. `dwa.update()`: kollisionsfreies Geschwindigkeitspaar wählen und als Radgeschwindigkeiten ausgeben.

3 Laufzeitumgebung und Projektstruktur

3.1 Werkzeuge

Element	Beschreibung
Simulator	Webots R2025b (Nightly-Build, Commit `033dbaea...`)
Sprache	C++ (objektorientiert, eine Klasse je Komponente)
Build	Webots-`Makefile.include`, je Controller ein `Makefile`
Robotermodell	`protos/coasterbot/Coasterbot.proto` (aus CAD abgeleitet)
Controller	`controllers/coasterbot/` (Quellcode)

Übersetzen und Ausführen:

```
export WEBOTS_HOME=~/Documents/webots/webots

# nur den Controller bauen
cd controllers/coasterbot && make

# eine Welt starten (GUI)
$\$WEBOTS_HOME/webots worlds/coasterbot-simtest.wbt

# headless start
$WEBOTS_HOME/webots --mode=fast --no-rendering --stdout --stderr \
    worlds/coasterbot-simtest.wbt
```

3.2 Quelldateien

Tabelle 5: Dateien des Controllers `coasterbot`.

Datei	Inhalt
`robot_hal.h`	abstraktes Interface `RobotHAL`, `enum WheelId`
`webots_hal.{h,cpp}`	`WebotsHAL : RobotHAL`, einzige von Webots-abhängige Datei
`pose_estimator.{h,cpp}`	`PoseEstimator`, Odometrie- und Gyro-Fusion
`occupancy_grid.{h,cpp}`	`OccupancyGrid`, Rasterkarte mit Distanzfeld
`astar.{h,cpp}`	`astarPlan`, `smoothPath`, `centerPath`
`dstar_lite.{h,cpp}`	`DStarLite`, inkrementeller Neuplaner
`dwa_planner.{h,cpp}`	`DwaPlanner`, lokale Planung und Bewegungssteuerung
`safety_monitor.{h,cpp}`	`SafetyMonitor`, Sicherheitskomponente
`robot_logic.{h,cpp}`	`RobotLogic`, reaktiver Zustandsautomat
`main.cpp`	Betriebsarten, Szenarien, Hauptschleifen
`Makefile`	Liste der zu übersetzenden Quelldateien

4 Hardwareabstraktion

4.1 Interface `RobotHAL`

`RobotHAL` (Datei `robot_hal.h`) ist eine abstrakte Klasse. Diese beschreibt *welche* Fähigkeiten der Roboter besitzt, jedoch nicht wie sie technisch realisiert werden. Die Klasse erfüllt `NFA-ARC-004` (eindeutig definierte Schnittstellen) und `NFA-ARC-005` (Hardwareabstraktion).

Tabelle 6: Methoden von `RobotHAL`.

Methode	Bedeutung / Einheit
`bool step()`	ein Regelzyklus; `false` = Simulation beendet
`void setLeftSpeed(float)`	Sollgeschwindigkeit linke Seite [rad/s], positiv = vorwärts
`void setRightSpeed(float)`	Sollgeschwindigkeit rechte Seite [rad/s]
`float getUltrasonicDistance()`	Abstand nach vorn [m], kleiner = näher
`bool getEdgeFrontLeft()` …	vier Kantensensoren; `true` = „kein Boden“ (Kante), bereits kalibriert
`float getYaw()`	Gierwinkel um die Hochachse [rad]
`float getGyroZ()`	Giergeschwindigkeit um die Hochachse [rad/s]
`float getWheelAngle(WheelId)`	akkumulierter Radwinkel EINER SEITE [rad], positiv = vorwärts
`float getTime()`	Laufzeit seit Programmstart [s]

Konventionen.

- **Voraussetzung:** `setLeftSpeed`/`setRightSpeed` positiv ⇒ Vorwärtsfahrt. Lenkung über die Drehzahldifferenz (Skid-Steer).
- **Radpaarung:** Die beiden Räder einer Seite sind mechanisch gekoppelt und laufen stets gemeinsam (wie der reale Coasterbot: zwei Getriebemotoren je Seite, gemeinsam angesteuert). Es gibt daher *keinen* Einzelrad-Befehl, `enum WheelId` hat deshalb nur die beiden Werte `WHEEL_LEFT`/`WHEEL_RIGHT`. Die reale Elektronik besitzt ebenfalls nur einen Encoder je Seite, nicht einen je Rad.
- **Kantensensoren:** `RobotHAL` gibt bereits ein kalibriertes `bool` zurück (`true` = Tischkante erreicht). Die Umrechnung des Rohwerts (in der Simulation ≈ 0 auf einer Fläche, ≈ 1000 über einer Kante; Schwelle 500) ist Aufgabe der jeweiligen HAL-Implementierung und bleibt in dieser gekapselt.

4.2 Implementierung `WebotsHAL`

`WebotsHAL` (Datei `webots_hal.cpp`) ist die Datei, welche die C++ API von Webots einbindet. Bei einer Portierung auf die Zielhardware kann diese Datei durch einen `ArduinoHAL` mit identischer Aufbau ersetzt werden, wobei die übrigen Komponenten unverändert bleiben.

Aufgaben de HAL:

- Beschaffung der Geräte-Handles über die im PROTO vergebenen Namen: die gekoppelten Motoren `wheel_left::front` und `wheel_right::front`, die vier PositionSensor `wheel_*_sensor` (je zwei pro Seite, siehe unten), der Ultraschall `ultrasonic`, die vier Kantensensoren `edge_*`, sowie `inertial_unit` und `gyro`. Die Namensstrings sind der Kopplung zwischen PROTO und Software.
- Betrieb der Antriebsmotoren über die Funktion (`setPosition(∞)`).
- **Vorzeichenkorrektur:** Das Radmodell des PROTO rollt den Roboter bei positiver Motordrehzahl nach $+Z$ (Rückseite). Da die die Vorderseite bei $-Z$ liegt, negiert `WebotsHAL` die Motor- und Encoder-Werte.
- **Gierrate:** Das Y-up-PROTO wird in der Z-up-Welt mit `rotation 1 0 0 1.5708` aufgestellt. Die Welt-Hochachse fällt daher mit der lokalen $+Y$-Achse des Gyros zusammen. Die Funktion `getGyroZ()` liefert die Y-Komponente des Gyrovektors.
- **Encoder-Aggregation:** Die reale Elektronik besitzt nur einen Encoder je Seite (`RobotHAL::WheelId` kennt deshalb nur `WHEEL_LEFT`/`WHEEL_RIGHT`). Das PROTO stellt in der Simulation trotzdem vier unabhängige `PositionSensor` bereit (je Rad). Die Funktion `getWheelAngle()` mittelt Front- und Heckrad je Seite, um nach außen dieselbe Information wie der einzelne reale Encoder zu liefern.
- **Digitales Kantensignal:** Die `lookupTable` der vier Kantensensoren im PROTO liefert ein digitales Signal (0 oder 1, keinen analogen Rohwert, siehe Abschnitt 13.1). Die Funktion `digitalEdge()` liest den Zustand und bildet ihn über die Konstante `EDGE_ACTIVE_HIGH` auf den `bool` Wert ab. An dieser Stelle muss im Code bekannt

sein, ob die „Kante“ am Sensor ein elektronisches HIGH oder LOW bedeutet. Die Funktion in `ArduinoHAL`, welche `digitalRead()` an einem Pin aufruft, passt bei einem aktiv-low-Sensormodul nur diese eine Konstante an.
- **Gyro-Simulation (GY-521/MPU-6050):** `getGyroZ()` stellt den idealen, physikalisch simulierten Wert Bias und das Rauschen des auf der Stückliste genannten Inertialsensors auf und kalibriert diesen Bias beim Start heraus. Details in Abschnitt 4.3.
- **Motorregelung Open-Loop/Closed-Loop:** `setLeftSpeed()` / `setRightSpeed()` setzen lediglich den Zustand. `updateMotorControl()` steuert die Motoren davon abhängig entweder direkt (Open-Loop) oder per Encoder-Rückkopplung geregelt (Closed-Loop, Standardeinstellung) an. Details und Vergleich in Abschnitt 4.4.

Die gekoppelten Motoren (coupled motors) von Webots setzen einen Befehl an `wheel_left::front` auch auf `wheel_left::rear` um (Faktor `multiplier` = 1), sodass die Software je Seite nur ein Handle benötigt.

4.3 Gyro-Simulation: GY-521/MPU-6050

Die Stückliste (BOM) des Projekts führt unter Position 11 einen Inertialsensor: „ICQUANZX GY-521 MPU6050; Navigationssensor“ (1 Stk., U12), ein GY-521-Breakout-Board mit dem InvenSense-MPU-6050-Chip (6-DOF Gyroskop und Beschleunigungssensor). Diese Festlegung gilt als gegeben für die reale Hardware. Deshalb bildet die Simulation diesen Chip nach, statt einen idealen, fehlerfreien Sensor anzunehmen.

Warum nicht über die PROTO-`lookupTable`. Für die vier Kantensensoren und den Ultraschall genügt Webots eingebauter `lookupTable`-Mechanismus (Abschnitt 13.1). Für den Gyro wurde dieser Weg geprüft und verworfen: Der dortige Rauschanteil skaliert mit dem Momentanwert der Messung und ist bei Stillstand (Messwert = 0) daher immer exakt null, ein realer Gyro rauscht aber auch im Stillstand messbar (Datenblattgröße „Total RMS Noise“). Dieses Verhalten wurde am simulierten Sensor überprüft (Roboter stationär: Rauschen exakt 0; Roboter drehend: Rauschen proportional zum Momentanwert), bevor die Simulation stattdessen software-seitig in `WebotsHAL` realisiert wurde. Die PROTO-`lookupTable` des Gyros bildet deshalb nur noch Messbereich und Auflösung ab (siehe Abschnitt 13.1); Bias und Rauschen kommen aus `WebotsHAL::getGyroZ()`.

Simulierte Fehlerquellen. Alle Werte sind über das MPU-6050-Datenblatt begründet (Werksdefault `FS_SEL` = 0, also $\pm 250\,^\circ\,\mathrm{s}^{-1}$ Messbereich, 16-Bit-ADC):

- **Nullpunktfehler (Zero-Rate Output):** Das Datenblatt nennt bis zu $\pm 20\,^\circ\,\mathrm{s}^{-1}$ Werkstoleranz ohne Kalibrierung. Da kein vermessenes reales Exemplar vorliegt, wird ein moderater, fest angenommener Demonstrationswert von $3\,^\circ\,\mathrm{s}^{-1}$ ($\approx 0.0524\,\mathrm{rad\,s}^{-1}$) simuliert. Dieser Wert ist eine Annahme und kein Messergebnis.

- **Rauschen:** Datenblattwert „Total RMS Noise“ $= 0.05\,^\circ/\mathrm{s}{-}\mathrm{rms}$ ($\approx 0.000\,873\,\mathrm{rad\,s^{-1}}$), als Standardabweichung eines Gauß-Rauschens mit festem statt zeitbasiertem Seed (`std::mt19937`, Seed 12345), damit `ST-SIM-004` reproduzierbar bleibt (siehe unten).
- **Messbereich und Auflösung:** in der PROTO-`lookupTable` (Abschnitt 13.1): $\pm 4.363\,323\,\mathrm{rad\,s^{-1}}$ ($= \pm 250\,^\circ\,\mathrm{s^{-1}}$), Auflösung $0.000\,133\,16\,\mathrm{rad\,s^{-1}}$ (16-Bit-ADC-Schrittweite).

Start-Kalibrierung. Ein reales MPU-6050-Modul wird üblicherweise beim Einschalten kalibriert, während das Gerät stillsteht. `WebotsHAL` bildet das nach: `calibrateGyro()` läuft im Konstruktor, *bevor* ein Fahrbefehl gegeben werden kann (die Motoren stehen zu diesem Zeitpunkt bereits auf `setVelocity(0.0)`), mittelt `GYRO_CAL_SAMPLES` $= 32$ Messwerte (bei 16 ms Regelzyklus ≈ 0.5 s) und zieht diesen Schätzwert von jeder folgenden Messung ab.

Diese Reihenfolge ist notwendig: Ein früherer Entwurf kalibrierte laufend während der ersten `getGyroZ()`-Aufrufe im laufenden Betrieb. In `MODE_SIM_TEST` beginnt die Navigation jedoch bereits im ersten Regelzyklus zu drehen. Die echte, sich ändernde Drehung floss dabei fälschlich in den Bias-Schätzwert ein und verfälschte ihn dauerhaft. Dies führte dazu, dass der Roboter in einer falschen Pose hängen blieb und keinen Testfall bestand. Die Kalibrierung *vor* dem ersten möglichen Fahrbefehl behebt diesen Fehler. Sie entspricht zugleich eher dem realen Vorbild (Kalibrierung beim Einschalten, nicht während der Fahrt).

Folge für die Lokalisierung. Weil nur `GYRO_CAL_SAMPLES` $= 32$ Messwerte gemittelt werden, bleibt ein Restfehler im Bias-Schätzwert zurück (Größenordnung $\sigma_{\mathrm{Rauschen}}/\sqrt{32} \approx 1.5 \times 10^{-4}\,\mathrm{rad\,s^{-1}}$). Da dieser Restfehler danach fest bleibt statt weiter nachgeführt zu werden, integriert `PoseEstimator` ihn über die Zeit zu einem langsamen Kursdrift auf, auch wenn der Roboter steht: gemessen $\approx -0.012\,^\circ\,\mathrm{s^{-1}}$ (Stillstandstest über 220 s, `MODE_ODOMETRY_TEST`). Für die kurzen Testläufe dieses Projekts (15 bis 25 s) ist das vernachlässigbar (siehe Abschnitt 5.4).

Verifikation. Nach dieser Änderung erneut gegen ST-SIM-001 bis ST-SIM-004 sowie `MODE_NAVIGATE_DSTAR` geprüft: weiterhin BESTANDEN bzw. korrektes Reroute, byte-identisch über zwei Läufe, Ziel- und Zeitwerte nur im Rahmen weniger Zehntelsekunden bzw. Millimeter verschoben. Bias und Rauschen bleiben also klein genug, um das bestehende Verhalten nicht zu verändern, machen die Simulation aber realitätsnäher.

4.4 Motorregelung: Open-Loop vs. Closed-Loop im Vergleich

Dieser Abschnitt setzt den in Abschnitt 4.2 bereits angekündigten Vergleich um: Beide Umsetzungen der Motoransteuerung (Open-Loop, Closed-Loop) wurden implementiert

und im selben Simulationslauf gegenübergestellt.

Warum ein reiner Vergleich in der Simulation nicht trivial ist. Das Motormodell im Geschwindigkeitsmodus (`Motor::setVelocity()`) wird in Webots *ideal simuliert*: Ein Kommando wird ohne Regelfehler exakt erreicht. Ohne eine zusätzliche, eingebaute Störgröße liefern Open-Loop und Closed-Loop deshalb in der Simulation exakt dasselbe Ergebnis und der Vergleich ist hinfällig. `WebotsHAL` prägt deshalb dem Kommando eine feste, *der Regelung unbekannte* Fertigungsstreuung zwischen den beiden Seiten auf (`MOTOR_GAIN_LEFT_TRUE` $= 0{,}95$, `MOTOR_GAIN_RIGHT_TRUE` $= 1{,}05$, also $\mp 5\,\%$): Ein Kommando von $4\,\mathrm{rad\,s^{-1}}$ führt an der realen (simulierten) Achse nicht zu exakt $4\,\mathrm{rad\,s^{-1}}$, sondern zu $4 \cdot \mathrm{Gain}$. Diese Werte sind eine Annahme (realistische Größenordnung für Getriebemotor-Toleranzen), kein Messwert eines realen Coasterbot-Exemplares.

Umsetzung in `WebotsHAL`. `setLeftSpeed()`/ `setRightSpeed()` setzen deshalb lediglich das *Kommando* (`commandedLeft_`/`commandedRight_`), wobei die eigentliche Aktuierung von `updateMotorControl()` übernommen wird. Dies geschieht einmal mit dem zuletzt gemessenen Encoderwert pro `step()`:

- **Gemessene Geschwindigkeit:** `measuredLeft_`/ `measuredRight_` aus der Änderung des jeweiligen Encoderwinkels (`RobotHAL::getWheelAngle`) seit dem letzten Regelzyklus, geteilt durch Δt. Über `measuredLeftSpeed()`/`measuredRightSpeed()` ist sie auch von außen abrufbar (nur für diesen Vergleich und kein Bestandeil der `RobotHAL`-Schnittstelle).
- **OPEN_LOOP:** `appliedLeft_`/`appliedRight_` (der tatsächlich an `Motor::setVelocity()` übergebene Wert) entsprechen unverändert dem Kommando. Der Gain-Fehler bleibt bestehen.
- **CLOSED_LOOP (Standardeinstellung):** `appliedLeft_`/ `appliedRight_` werden jeden Regelzyklus per Integralregler nachgeführt,

  $$\text{applied} \mathrel{+}= K_I \cdot (\text{commanded} - \text{measured}) \cdot \Delta t, \qquad K_I = 25,$$

  begrenzt auf $\pm 20\,\mathrm{rad\,s^{-1}}$ (Anti-Windup, entspricht der PROTO-`maxVelocity`). Die Regelung kennt den Gain-Fehler nicht, gleicht ihn aber allein über die Rückkopplung aus.

`CLOSED_LOOP` ist die **Standardeinstellung**, damit die anderen Betriebsarten (`MODE_SIM_TEST`, `MODE_NAVIGATE_DSTAR`, ...) ihr bereits verifiziertes Verhalten beibehalten. Nur die neue Betriebsart `MODE_MOTOR_CONTROL_TEST` schaltet bewusst auf `OPEN_LOOP` um, um den Unterschied zu zeigen.

Vergleichstest `MODE_MOTOR_CONTROL_TEST`**.** Auf der offenen Fläche (`coasterbot-testfield.wbt`) kommandiert der Test beiden Seiten denselben Geradeausbefehl ($4\,\mathrm{rad\,s^{-1}}$), zunächst 8 s lang Open-Loop, danach 8 s lang Closed-Loop (derselbe Lauf, kein Zurücksetzen der Position nötig: Kursdrift und Geschwindigkeitsfehler werden je Phase relativ zum Phasenanfang gemessen). Bei identischem Kommando auf beiden Seiten sollte der Roboter nicht abdriften.

Verifiziertes Ergebnis eines simulierten Testlaufs:

Messgröße	**Open-Loop**	**Closed-Loop**
Kursdrift über 8 s	24.9°	0.05°
Geschwindigkeitsfehler links (Phasenende)	$0.20\,\mathrm{rad\,s^{-1}}$	$< 0.001\,\mathrm{rad\,s^{-1}}$
Geschwindigkeitsfehler rechts (Phasenende)	$-0.20\,\mathrm{rad\,s^{-1}}$	≈ 0

Der gemessene Geschwindigkeitsfehler im Open-Loop-Fall ($\approx \pm 5\,\%$ von $4\,\mathrm{rad\,s^{-1}}$) entspricht exakt dem eingeprägten Gain-Fehler. Die Closed-Loop-Regelung konvergiert innerhalb weniger Regelzyklen (deutlich unter 1 s) auf den Sollwert und hält den Kursdrift auf einer Größenordnung, welcher auch ohne jeden Gain-Fehler allein durch das Restrauschen des simulierten Gyros (Abschnitt 4.3) zu erwarten ist.

Verifikation der Rückwärtskompatibilität. Da `CLOSED_LOOP` nun in *allen* Betriebsarten aktiv ist (vorher war die Motoransteuerung effektiv ideal), wurde die bestehende Testreihe erneut geprüft: `MODE_SIM_TEST` bleibt ST-SIM-001 bis ST-SIM-004 BESTANDEN, `MODE_NAVIGATE_DSTAR` reroutet weiterhin korrekt, in beiden Fällen nur geringfügig andere Zeit-/Wegpunkte als in Abschnitt 4.3 (der Integralregler führt eine kurze, aber echte Regeldynamik ein, wo zuvor eine ideale Momentanantwort stand). Alle Erfolgskriterien bleiben unverändert erfüllt.

5 Lokalisierungskomponente

Die Lokalisierung (`PoseEstimator`, Dateien: `pose_estimator.{h,cpp}`) bestimmt fortlaufend die Pose des Roboters in der Tischebene und erfüllt `NAV-006` (Positionsbestimmung) und `NAV-007` (Orientierungsbestimmung).

5.1 Prinzip

Der Coasterbot *misst* seine Position nicht, sondern **führt sie durch Koppelnavigation (Dead Reckoning) fort**. Es gibt keine externe Referenz: kein GPS, Bake, oder kamerabasierte Lokalisierung. Die Pose entsteht in vier Schritten:

1. **Bekannte Startpose.** `main.cpp` setzt den Schätzer über `pose.reset(x, y, θ)` auf die im Szenario/in der Welt definierte Startposition. Alles Weitere ist relativ zu diesem Punkt.
2. **Radodometrie.** In jedem Regelzyklus wird über `RobotHAL::getWheelAngle` je ein Encoderwert pro Seite gelesen (`WHEEL_LEFT`/`WHEEL_RIGHT`, die reale Elektronik hat nur einen Encoder je Seite) und mit dem Radradius in eine gefahrene Strecke umgerechnet.
3. **Kursfusion.** Die Kursänderung wird aus der Radodometrie *und* der integrierten Gyro-Giergeschwindigkeit über einen Komplementärfilter bestimmt. Der Gyro wird dabei vorrangig behandelt.
4. **Integration.** Strecke und Kurs werden zur neuen Pose (x, y, θ) aufsummiert.

5.2 Verfahren

Verwendet wird eine **Skid-Steer-Odometrie** aus den zwei seitenweisen Radwinkeln, deren Kurs mit der integrierten Gyro-Giergeschwindigkeit über einen **Komplementärfilter** fusioniert wird. Die Alternativenbeschreibung nennt hierfür grundsätzlich EKF/UKF. Für den Prototyp, der auf einer Tischfläche steht und mit nur zwei Sensorquellen ausgestattet ist, genügt ein wesentlich einfacherer Komplementärfilter. Dieser kommt ohne Kovarianzschätzung und ist deterministisch (wichtig für `ST-SIM-004`).

Aus dem jeweiligen Encoderwert folgt die gefahrene Strecke einer Seite (in der Simulation mittelt `WebotsHAL` dafür intern die zwei PositionSensor von Front- und Heckrad derselben Seite, um dieselbe Information wie der einzelne reale Encoder je Seite nachzubilden):

$$\Delta s_L = \big(\varphi_L - \varphi_{L,\text{prev}}\big)\, r, \qquad \Delta s_R = \big(\varphi_R - \varphi_{R,\text{prev}}\big)\, r$$

mit Radradius $r = 0.035\,\text{m}$. Daraus folgen der Vorschub des Zentrums und die odometrische Kursänderung:

$$\Delta s = \tfrac{1}{2}\big(\Delta s_L + \Delta s_R\big), \qquad \Delta\theta_{\text{odo}} = \frac{\Delta s_R - \Delta s_L}{b}, \qquad b = 0.202\,\text{m (Spurweite)}.$$

Die fusionierte Kursänderung gewichtet Odometrie und Gyro:

$$\Delta\theta = w\,\Delta\theta_{\text{odo}} + (1 - w)\,\dot{\psi}_{\text{gyro}}\,\Delta t, \qquad w = 0{,}05.$$

Da Skid-Steer beim Drehen stark schlupft, wird überwiegend dem Gyro vertraut ($w = 0{,}05$). `getGyroZ()` liefert dabei die Y-Komponente des Gyrovektors: Das Y-up-PROTO wird in der Z-up-Welt mit `rotation 1 0 0 1.5708` aufgestellt, wodurch die lokale $+Y$-Achse mit der Welt-Hochachse der Simulation identisch ist.

Die Integration erfolgt über den mittleren Kurs des Schritts (Runge-Kutta 2. Ordnung):

$$x \mathrel{+}= \Delta s\,\cos\!\big(\theta + \tfrac{1}{2}\Delta\theta\big), \quad y \mathrel{+}= \Delta s\,\sin\!\big(\theta + \tfrac{1}{2}\Delta\theta\big), \quad \theta \mathrel{+}= \Delta\theta.$$

5.3 Schnittstelle

Methode	Bedeutung
`void update()`	einmal pro Regelzyklus nach `hal.step()`
`float getX() / getY()`	Position [m]
`float getTheta()`	Kurs [rad], $(-\pi, \pi]$
`float getOdometer()`	gesamte gefahrene Strecke [m]
`void reset(x,y,`θ`)`	Pose auf bekannten Startwert der Welt setzen

Koordinatenkonvention: x nach vorn (Startausrichtung), y nach links, θ positiv gegen den Uhrzeigersinn. Der absolute Gierwinkel der InertialUnit (`getYaw()`) wird im Schätzer bewusst *nicht* verwendet, da die 90°-Aufstellrotation die IMU nahe den Gimbal-Lock in der Nickachse bringt.

5.4 Grenzen der Lokalisierung

- **Driftakkumulation.** Reine Koppelnavigation summiert Messfehler auf. Ohne absolute Korrektur oder Schleifenschluss (loop closure) wächst der Positionsfehler mit der gefahrenen Strecke. Für die kleine Tischfläche und die kurzen Fahrten der Simulationstests ist das ausreichend.
- **Manuelles Umsetzen.** Wird der Roboter physisch angehoben und versetzt, ist die Schätzung falsch, bis `reset()` erneut eine bekannte Pose vorgibt, entsprechend der Einschränkung in `NAV-006` („solange er nicht manuell umgesetzt wurde“).
- **Verifikation.** `MODE_ODOMETRY_TEST` auf `coasterbot-testfield.wbt`: Geradeausfahrt 0.699 m gegenüber 0.70 m Soll, kein seitlicher Versatz, Drehung auf der Stelle wird vom Gyro sauber verfolgt. Im Stillstand bleibt die Pose nahezu stabil, mit dem seit Abschnitt 4.3 simulierten Gyro-Restfehler driftet der Kurs im Stillstand um $\approx 0.012\,^\circ\,\mathrm{s}^{-1}$. Dies ist für die kurzen Testläufe dieses Projekts vernachlässigbar.

6 Umgebungsmodell

`OccupancyGrid` (Dateien `occupancy_grid.{h,cpp}`) entspricht der im Pflichtenheft genannten „bekannten Tischkarte“ und ist die gemeinsame Eingabe für A*, D* Lite und DWA. Es setzt `SIM-004` um (Hindernisse in der Simulation berücksichtigen).

6.1 Aufbau

- Achsenparalleles Rechteck in **Weltkoordinaten** (X-Y-Ebene der Webots-Welt, Z = oben) mit fester Auflösung (Standard 0.05 m). Es liegt in derselben Ebene wie die Pose.

- In den Simulationsszenarien ist das Gitter auf die Tischfläche begrenzt, damit geplante Wege die Tischkante nicht überschreiten können.
- Zellen sind *frei* oder *belegt*. Zellen außerhalb der Karte gelten als belegt.

6.2 Funktionen

Funktion	Wirkung
`addObstacleRect(..., infl)`	Rechteck-Hindernis, um `infl` Meter einzutragen (Roboter-Halbbreite ⇒ Punktroboter-Planung, erfüllt `NAV-010`)
`setOccupied(Cell,bool)`	einzelne Zelle mit (hartem) Wert setzen. Rückgabe `true` bei Änderung (für inkrementelle Neuplanung)
`reportOccupied(Cell, smoothingFactor)`	einzelne Zelle GEGLÄTTET melden (Abschnitt 6.3)
`buildDistanceField()`	Chamfer-Distanzfeld über zwei Rasterdurchläufe berechnen
`clearance(Vec2)`	Abstand eines Weltpunkts zum nächsten Hindernis [m]
`worldToCell / cellToWorld`	Umrechnung Welt ↔ Zelle

Das Distanzfeld wird nach jeder Hindernisänderung als veraltet markiert und muss vor der nächsten `clearance()`-Abfrage neu gebaut werden. In `MODE_SIM_TEST` und `MODE_NAVIGATE_DSTAR` werden *zwei* Gitter geführt: ein großzügiges aufgespanntes Planer-Gitter (hält den Pfad von Ecken fern) und ein nur um den Roboterradius aufgespanntes DWA-Gitter (Kollisionsbewertung). Beide werden bei Sensortreffern gemeinsam aktualisiert.

6.3 Geglättete Live-Hindernismeldung

`addObstacleRect()` trägt a priori bekannte Hindernisse fest und sofort ein (z. B. die bekannte Bierflasche im Simulationstest). Dies ist machbar, weil diese Eingabe nicht verrauscht ist. Ein *live* erkanntes per Kantensensor gemessenes Hindernis, während der Fahrt, kann durch Rauschen oder eine kurze Fehlmessung verfälscht werden. Bis Version 5 dieses Dokuments hätte bereits ein einzelner solcher Treffer die Karte sofort verändert (`setOccupied(c, true)`) und damit eine D*-Lite-Umplanung ausgelöst.

`reportOccupied(Cell, smoothingFactor)` glättet das: statt die Zelle sofort zu belegen, wird eine Belegungs-*Konfidenz* conf $\in [0, 1]$ je Zelle exponentiell nachgeführt,

$$\text{conf} \mathrel{+}= \text{smoothingFactor} \cdot (1 - \text{conf}),$$

und die Zelle erst dann tatsächlich belegt (wie `setOccupied`), wenn conf $\geq$ `OCC_COMMIT_THRESHOLD` (Anhang B). Mit dem Standardwert `smoothingFactor` = 0,5 genügen bereits drei aufeinanderfolgende Treffer an derselben Zelle (bei 32 ms Regelzyklus also $\approx$ 100 ms), ein einzelner Ausreißer reicht dagegen nicht aus. Die Datei `main.cpp` verwendet `reportOccupied()` inzwischen dort, wo bisher `setOccupied()` für Live-Sensortreffer aufgerufen wurde (`dstar_scenario::markRect`, `sim_scenario::markRect`).

Die Reflexbewegung bleibt davon unberührt: `DwaPlanner` bewertet die Kollisionsfreiheit eines Kurses weiterhin in *jeden* Zyklus direkt aus dem rohen ungeglätteten Messwert als virtuellem Hindernispunkt (Abschnitt 9.3). Geglättet wird ausschließlich die Entscheidung, ob D* Lite die globale Karte dauerhaft ändert und umplant. Die sofortige Ausweichreaktion bei akuter Kollisionsgefahr bleibt so unverzögert.

Verifiziert: `MODE_SIM_TEST` und die Hinderniserkennung in `MODE_NAVIGATE_DSTAR` liefern nach der Umstellung die Ergebnisse wie zuvor (ST-SIM-001..004 weiterhin BESTANDEN, byte-identisch über zwei Läufe, D*-Lite-Umplanung in `coasterbot-dstar.wbt` weiterhin bei $t \approx 3.8$ s, Ziel nach 24 s mit 1.5 cm Restfehler). Die zusätzliche Latenz durch die Glättung liegt unterhalb der Auflösung dieser Szenarien.

7 Globale Pfadplanung mit A*

`astarPlan(grid, start, goal)` (Dateien `astar.{h,cpp}`) berechnet einen globalen Fahrweg auf der bekannten Karte. Umgesetzte Anforderungen: `NAV-008` (Zielpunkt erreichen), `NAV-009` (autonome Bewegung), `NAV-010` (Sicherheitsabstand).

7.1 Algorithmus

- 8-fach verbundenes Gitter, euklidische Heuristik, Prioritätswarteschlange.
- Kein „corner cutting“: eine Diagonale ist nur befahrbar, wenn auch die beiden orthogonalen Nachbarzellen frei sind.
- Liegen Start oder Ziel in einer aufgebauten Hinderniszone, wird per Ringsuche die nächste freie Zelle verwendet.
- **Glättung („string pulling“):** `smoothPath()` behält von jedem Anker den entferntesten noch per Sichtlinie erreichbaren Wegpunkt. Das vermeidet die 45°-Treppen des Rohpfades.
- **Zentrierung:** `centerPath()` verschiebt innere Wegpunkte senkrecht zur Fahrtrichtung in Richtung eines größeren Hindernisabstands (max. 0.20 m), damit der Pfad in engen Korridoren mittig verläuft. Ohne diesen Schritt bleibt der DWA-Planer an eckennahen Pfaden hängen.

Rückgabe: Wegpunkte in Weltkoordinaten von Start bis Ziel. Ein leerer Vektor bedeutet „kein Pfad“.

7.2 Auswahlbegründung

Gemäß Alternativenbeschreibung standen Dijkstra und BFS als Alternativen zur Wahl. A* wurde wegen der zielgerichteten Heuristik gewählt: Bei gleichem optimalem Zustand (zulässige Heuristik) expandiert A* deutlich weniger Knoten als Dijkstra. Für die kleine Tischkarte ($\sim 32 \times 40$ Zellen) ist die Erstplanung damit in weniger als einer Millisekunde abgeschlossen.

8 Dynamische Neuplanung mit D* Lite

`DStarLite` (Dateien `dstar_lite.{h,cpp}`) repariert den Pfad inkrementell, wenn während der Fahrt ein Hindernis auftaucht oder verschwindet. Umgesetzte Anforderung: `NAV-005` (Fahrtroute während der Bewegung anpassen).

8.1 Verfahren

Implementiert ist D* Lite nach Koenig & Likhachev (2002):

- Rückwärtssuche vom Ziel. Je Zelle ein g-Wert (bekannte Kosten) und ein rhs-Wert („one-step lookahead“). Eine Zelle ist konsistent, wenn $g = rhs$.
- `std::set`-basierte Prioritätswarteschlange über Schlüssel (k_1, k_2); der Modifikator k_m akkumuliert die Heuristikverschiebung, während der Roboter fährt, sodass die Warteschlange ohne vollständige Neusortierung gültig bleibt.
- `plan(start, goal)` führt eine *vollständige* Erstexpansion der erreichbaren Karte durch (`expandAll`). Ein vollständiges g-Feld sorgt dafür, dass eine spätere lokale Änderung auch dann repariert wird, wenn der Ausweichweg zuvor nicht „auf dem Weg“ lag.
- `setStart(cell)` bei jedem Zellwechsel des Roboters. `cellChanged(cell)` bei jeder Belegungsänderung stellt die betroffenen Knoten neu in die Warteschlange. Die Funktion `computeShortestPath()` repariert dann nur den betroffenen Bereich.
- `extractPath()` folgt vom Start greedy den kleinsten g-Werten zum Ziel. `pathExists()` und `finiteGCount()` dienen der Diagnose.
- Ist der gesamte geplante Korridor durchtrennt (inkrementelle Reparatur liefert keinen Pfad), setzt `main.cpp` D* Lite aus der aktuellen Pose vollständig neu auf, was günstiger ist als eine Neuplanung in jedem Zyklus.

Alternative laut Alternativenbeschreibung war LPA*. D* Lite ist die auf bewegliche Startpositionen zugeschnittene Variante von LPA* und daher für einen fahrenden Roboter die passende Wahl.

9 Lokale Kollisionsvermeidung und Bewegungssteuerung mit DWA

`DwaPlanner` (Dateien `dwa_planner.{h,cpp}`) folgt dem globalen Pfad und wählt in jedem Regelzyklus ein sicheres Geschwindigkeitspaar (v, ω). Er vereint die Pflichtenheft-Komponenten „lokale Kollisionsvermeidung“ und „Bewegungssteuerung“ und setzt `NAV-001`, `NAV-002`, `SAF-003` und `SAF-004` um.

9.1 Dynamic Window Approach

1. **Dynamisches Fenster:** Aus der aktuellen Geschwindigkeit und den Beschleunigungsgrenzen ($a_v = 0.5\,\mathrm{m\,s^{-2}}$, $a_\omega = 6\,\mathrm{rad\,s^{-2}}$) wird der in einem Zeitschritt erreichbare (v, ω)-Bereich bestimmt und in 6×25 Kandidaten abgetastet. Grenzen: $v \in [0,\ 0.18\,\mathrm{m\,s^{-1}}]$, $|\omega| \le 2.6\,\mathrm{rad\,s^{-1}}$.
2. **Vorwärtssimulation:** Jeder Kandidat wird über $T_{\mathrm{sim}} = 1.5\,\mathrm{s}$ (Schritt $0.1\,\mathrm{s}$) kinematisch simuliert.
3. **Verwerfen:** Kurse mit Kollision oder solche, aus denen der Roboter nicht mehr bremsen kann ($v > \sqrt{2\,a_v\,\mathrm{clearance}}$), werden verworfen. Damit ist `SAF-004` (keine schädigende Anfahrgeschwindigkeit) erfüllt.
4. **Bewertung:** Die verbleibenden Kandidaten werden gewichtet bewertet:

$$S = w_h\,S_{\mathrm{Kurs}} + w_p\,S_{\mathrm{Fortschritt}} + w_c\,S_{\mathrm{Abstand}} + w_v\,S_{\mathrm{Tempo}}$$

   mit $w_h = 1{,}0$, $w_p = 2{,}0$, $w_c = 1{,}5$, $w_v = 0{,}3$. S_{Kurs} bewertet die Ausrichtung zum Carrot-Punkt, $S_{\mathrm{Fortschritt}}$ die Annäherung an ihn, S_{Abstand} den Hindernisabstand entlang der Trajektorie, S_{Tempo} die Fahrgeschwindigkeit.
5. **Sackgasse:** Ist kein Kandidat zulässig, dreht der Roboter langsam auf der Stelle, bis wieder Optionen entstehen.

9.2 Pfadverfolgung (Pure Pursuit)

Der **Carrot-Punkt** ist der Punkt $0.30\,\mathrm{m}$ entlang des Polygonzugs vor der Projektion der Roboterposition auf den Pfad. Da er stets auf dem kollisionsfrei geglätteten Pfad liegt, bleibt die globale Hindernisumfahrung erhalten, während DWA die lokale, dynamikkonforme Ausführung übernimmt.

9.3 Hindernisse

DWA bezieht Hindernisse aus *zwei* Quellen:

- dem **Distanzfeld** der (bereits ausgelegten) Belegungskarte: hier wird nur ein kleiner Restpuffer `GRID_MARGIN` $= 0.03\,\mathrm{m}$ abgezogen.

- dem **aktuellen Messwert** als virtuellem Hindernispunkt vor dem Roboter: ein roher Punkt, von dem der volle Roboterradius 0.14 m abgezogen wird. Damit werden unkartierte bzw. dynamische Hindernisse abgedeckt (`NAV-001`).

Über `setUltrasonicRange(r)` lässt sich der Ultraschalleinfluss begrenzen: Trägt eine übergeordnete Ebene Hindernisse selbst in die Karte ein, wirkt DWA nur noch als Nahbereichs-Sicherung (`MODE_SIM_TEST`: $r = 0.45\,\mathrm{m}$).

9.4 Bewegungssteuerung

`driveVW(v, ω)` rechnet das gewählte Geschwindigkeitspaar in Radgeschwindigkeiten um

$$\omega_L = \frac{v - \omega\, b/2}{r}, \qquad \omega_R = \frac{v + \omega\, b/2}{r}$$

($r = 0.035\,\mathrm{m}$, $b = 0.202\,\mathrm{m}$) und gibt sie über `RobotHAL::setLeftSpeed` / `setRightSpeed` aus. Die Geschwindigkeitsbegrenzung (MAX_V), das kontrollierte Anhalten am Ziel (`GOAL_TOL` $= 0.05\,\mathrm{m}$) und der sofortige Stopp bei fehlendem Pfad decken die im Pflichtenheft geforderten Eigenschaften der Bewegungssteuerung ab.

Alternative laut Alternativenbeschreibung waren Potentialfelder. Diese wurden wegen der möglichen lokalen Minima nicht gewählt. DWA umgeht das Problem durch die explizite Trajektorienbewertung und die Kopplung an den globalen Pfad.

10 Sicherheitskomponente

`SafetyMonitor` (Dateien `safety_monitor.{h,cpp}`) ist die im Lasten- und Pflichtenheft geforderte **Sicherheitskomponente / Sicherheitslogik** mit übergeordneter Priorität. Umgesetzte Anforderungen: `NAV-003`, `NAV-004`, `SAF-001`, `SAF-002` (Struktur), `SAF-003`, `SAF-005`.

10.1 Funktion

In jedem Regelzyklus (*vor* der Navigation) wertet der Monitor die vier Kantensensoren aus. Erkennt ein Sensor die Tischkante, hält er sofort an, übernimmt die Aktorik und führt eine Ausweichbewegung aus:

Situation	**Reaktion**
Kante vorne	Rückwärtsfahrt (`EVADE_RADS` $= 3\,\mathrm{rad\,s^{-1}}$, `EVADE_TIME` $= 0.7\,\mathrm{s}$)
Kante hinten	Vorwärtsfahrt
Kante vorne *und* hinten	Stillstand (Zustand: `PINNED`)

Zustandsautomat: CLEAR → EVADE → PAUSE (0.3 s) → CLEAR, bei beidseitiger Kante PINNED. Solange die Funktion overriding() true ist, darf die Navigation *keine* eigenen Fahrbefehle geben. Dies wird in main.cpp sichergestellt, indem der Regelzyklus nach einem haltenden safety.update() vorzeitig endet.

10.2 Schnittstelle

Methode	Bedeutung
bool update()	true, solange die Komponente die Aktorik hält
bool overriding()	Zustand ≠ CLEAR
bool edgeEventThisTick()	steigende Flanke einer Kantenerkennung
bool frontEdge() / rearEdge()	betroffene Seite
const char* stateName()	aktueller Zustand (Systemüberwachung, MON-003)
int eventCount()	Zähler aller Kantenereignisse

Not-Aus (SAF-005). Der Software-seitige Anker ist die sofortige Aktorübernahme mit drive(0). Der physische Not-Aus-Taster (HW-BTN-003) wird über dieselbe Abstraktions-Schnittstelle als Ereignis eingespeist, ist jedoch in der Simulation nicht modelliert.

Sensorausfall (SAF-002). Die Zustandsstruktur für den Wechsel in einen sicheren Zustand ist vorhanden; die gezielte Fehlerinjektion (ST-SIM-005) übernimmt FaultMonitor, siehe Abschnitt 10.3.

10.3 Fehlerüberwachung (FaultMonitor)

FaultMonitor (Dateien fault_monitor.{h,cpp}) deckt die Fehlerinjektionstests ST-SIM-005 (Sensorausfall) und ST-SIM-006 (Aktorfehler) ab. Wie SafetyMonitor kennt er nur RobotHAL und ist damit Arduino-portabel; anders als SafetyMonitor ist sein sicherer Zustand **verriegelt**: ein Sensor-/Aktorausfall behebt sich nicht von selbst, es gibt also keine automatische Rückkehr nach NORMAL.

Prüfung	Kriterium
Ultraschall (`SENSOR_FAULT`)	Messwert außerhalb $[0, $`ULTRASONIC_MAX_VALID`$]$. Die Schwelle liegt bei 4.5 m, mit deutlicher Marge zum vom PROTO simulierten Sensorrauschen (die `lookupTable` trägt am oberen Ende 2 % Rauschen auf 4 m, einzelne Messungen liegen daher legitim leicht über 4 m - beobachtet bis 4.04 m). Eine zu enge Schwelle hätte hier ganz normales Sensorrauschen fälschlich als Ausfall gemeldet
Antrieb (`ACTUATOR_FAULT`)	kommandierte vs. aus `getWheelAngle()` gemessene Radgeschwindigkeit einer Seite weicht um mehr als `SPEED_DEVIATION_RADS` ($1\,\mathrm{rad\,s^{-1}}$) ab, bei einem Kommando über `MIN_COMMANDED_RADS` ($0.5\,\mathrm{rad\,s^{-1}}$), anhaltend über `FAULT_DEBOUNCE_TIME` (0.3 s) - die Debounce-Zeit verhindert Fehlalarm durch die normale Beschleunigungsrampe

Bestätigt sich ein Fehler, übernimmt `FaultMonitor` sofort die Aktorik mit `drive(0)` und hält sie, genau wie `SafetyMonitor`: `bool update(commandedLeft, commandedRight)` liefert `true`, solange die Komponente die Aktorik hält, der Aufrufer darf dann keine eigenen Fahrbefehle mehr geben.

Die Fehler selbst werden simulationsseitig in `WebotsHAL` erzeugt (`injectUltrasonicFault()` / `injectLeftMotorFault()`) - wie `MotorControlMode` (Abschnitt 4.4) sind das reine Simulationsschalter, *nicht* Teil von `RobotHAL`: ein realer Ausfall entsteht durch Hardware, nicht durch einen Laufzeit-Schalter. `FaultMonitor` sieht nur das Symptom über die normale `RobotHAL`-Schnittstelle - einen Ultraschallwert außerhalb des Kontrakts bzw. ein linkes Rad, das trotz Kommando nicht dreht - und kann den simulierten Fehler daher nicht von einem echten unterscheiden.

10.4 Ablauf `MODE_FAULT_INJECTION`

Zwei Phasen auf offener Fläche (`coasterbot-testfield.wbt`, keine Kanten/Hindernisse, damit ausschließlich der injizierte Fehler das Ergebnis bestimmt), je mit einem frisch instanzierten `FaultMonitor`: Geradeausbefehl für 3 s, danach den jeweiligen Fehler über `WebotsHAL` auslösen und messen, ob/wann der Monitor reagiert.

1. **Phase 1 (`ST-SIM-005`):** Ultraschall auf nicht verfügbarßchalten. Verifiziert (kopfloser Lauf, `coasterbot-testfield.wbt`): Erkennung binnen eines Regelzyklus (< 0.02 s), Aktorik danach nachweislich im Stillstand (gemessene Radgeschwindigkeit $\approx 0\,\mathrm{rad\,s^{-1}}$).

2. **Phase 2 (ST-SIM-006):** linker Motor fällt aus (reagiert nicht mehr auf Kommandos). Verifiziert: Erkennung nach 0.34 s (knapp über der Debounce-Zeit, wie vorgesehen), Aktorik danach im Stillstand.
3. Bewertung je Testfall (BESTANDEN, wenn Fehler erkannt *und* die Aktorik anschließend nachweislich gestoppt bleibt). Zwei kopflose Läufe lieferten identische Erkennungszeiten und Verdikte (ST-SIM-004-Prinzip).

11 Reaktive Verhaltenslogik

RobotLogic (Dateien robot_logic.{h,cpp}) ist ein reaktiver Zustandsautomat zur Hindernis- und Kantenvermeidung. Er dient der Inbetriebnahme der HAL und ist die Grundlage der Betriebsart MODE_AUTONOMOUS. Für die zielgerichtete Navigation wird er durch den Navigationsstapel (A* / D* Lite / DWA) ersetzt.

Zustand	Verhalten
FORWARD	geradeaus mit CRUISE_SPEED = $6\,\mathrm{rad\,s^{-1}}$; bei Kante → REVERSE, bei Hindernis ($< 0.22\,\mathrm{m}$) → AVOID_OBSTACLE
REVERSE	0.6 s zurücksetzen, dann TURN
TURN	auf der Stelle drehen (zufällige Richtung), bis frei
AVOID_OBSTACLE	drehen, bis der Sensor frei ist

Zusätzlich stellt RobotLogic die expliziten Bewegungsprimitive forward, backward, turnLeft, turnRight und stop bereit, die in den Test-Betriebsarten direkt aufgerufen werden.

12 Ablaufsteuerung und Betriebsarten

Die Anwendungsebene (main.cpp) instanziiert WebotsHAL, RobotLogic und PoseEstimator und wählt über den Kompilierschalter static const Mode MODE eine Betriebsart. So ist nachvollziehbar, welche Bewegung warum ausgeführt wird (NFA-TST-*: gezielt erzeugbare, reproduzierbare Abläufe).

Tabelle 14: Betriebsarten des Controllers.

MODE	Zweck	Welt
MODE_WHEEL_TEST	je Seite 2 s die Räder drehen, Encoder ausgeben	beliebig
MODE_MANUAL	feste Befehlsfolge über die Bewegungsprimitive	beliebig

MODE	Zweck	Welt
MODE_AUTONOMOUS	reaktiver Zustandsautomat RobotLogic	coasterbot.wbt
MODE_ODOMETRY_TEST	definierte Fahrt, Pose ausgeben (Odometrieprüfung)	coasterbot-testfield.wbt
MODE_NAVIGATE	A*-Pfad planen und mit DWA abfahren	coasterbot-astar.wbt
MODE_NAVIGATE_DSTAR	D* Lite + per Ultraschall entdecktes Hindernis → Umplanung	coasterbot-dstar.wbt
MODE_SIM_TEST	kombinierter Simulationstest, Bewertung ST-SIM-001..004	coasterbot-simtest.wbt
MODE_MOTOR_CONTROL_TEST	Vergleich Open-Loop/Closed-Loop-Motorregelung (Abschnitt 4.4)	coasterbot-testfield.wbt
MODE_FAULT_INJECTION	Fehlerinjektionstest, Bewertung ST-SIM-005/006 (Abschnitt 10.4)	coasterbot-testfield.wbt

12.1 Ablauf MODE_SIM_TEST

1. Pose auf die Startpose der simulierten Welt setzen. Planer- und DWA-Gitter aus dem Szenario sim_scenario aufbauen (Tischrand als Absturzkante, bekannte Bierflasche auf der Route, Deko in den Ecken).
2. D* Lite Erstplanung, Pfad glätten und zentrieren.
3. **Phase 1 (Navigation, ST-SIM-001/002):** Sicherheitskomponente → Kartenpflege → D* Lite → DWA, bis das Ziel innerhalb der Toleranz erreicht ist. Kleinster Hindernisabstand und Anzahl der Umplanungen werden protokolliert.
4. **Phase 2 (Tischkante, ST-SIM-003):** Roboter richtet sich auf $+Y$ aus und fährt langsam Richtung Tischkante. Danach darf nur die Sicherheitskomponente eingreifen. Es wird gezählt, wie oft sie die Kante erkennt und ob der Roboter auf dem Tisch bleibt.
5. Bewertung ausgeben (BESTANDEN / FEHLGESCHLAGEN je Testfall). ST-SIM-004 wird durch zweimaligen Lauf und Vergleich der Ausgabe geprüft (byte-identisch, da alle Komponenten deterministisch sind).

12.2 Vorschlag: Missionsablaufsteuerung

Die Variable MODE wählt eine *Testbetriebsart* für die Simulation. Sie ist kein Ersatz für die im Pflichtenheft geforderte *Missionsablaufsteuerung* des realen Prototyps (IDLE →

... → `DONE`, siehe `Pflichtenheft/Zustandsautomaten.svg`), die noch nicht implementiert ist. Abbildung 3 zeigt einen dafür ausgearbeiteten **Aufbau** (Stand 22.09.2026), der die abstrakten Zustände aus dem Pflichtenheft durch die konkreten Betriebszustände des Prototyps ersetzt.

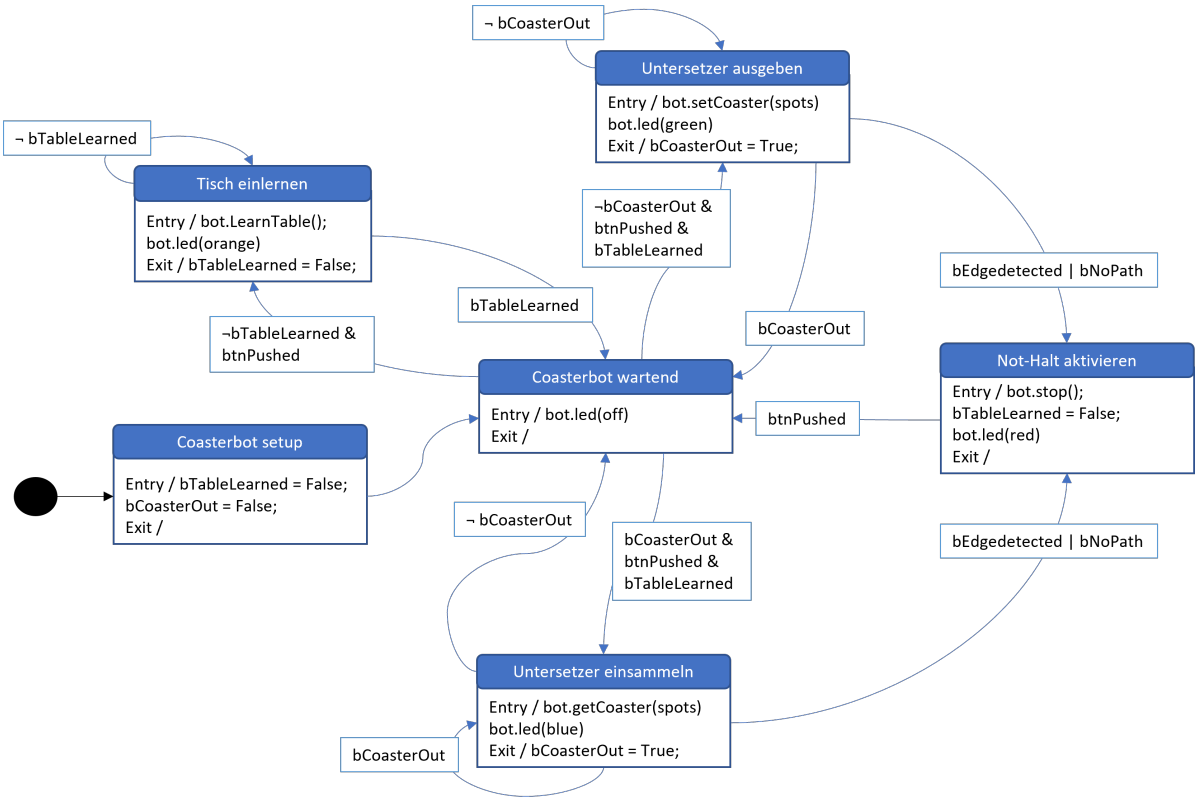


Abbildung 3: Ablauf für die Missionsablaufsteuerung des Coasterbot-Prototyps: sechs Zustände (*Coasterbot setup*, *Coasterbot wartend*, *Tisch einlernen*, *Untersetzer ausgeben*, *Untersetzer einsammeln*, *Not-Halt aktivieren*) mit Entry-/Exit-Aktionen (LED-Farbe, `bot.LearnTable()`, `bot.setCoaster()`, `bot.getCoaster()`, `bot.stop()`) und Wächterbedingungen auf den Flags `bTableLearned`, `bCoasterOut`, `btnPushed` sowie den Fehlersignalen `bEdgedetected`/`bNoPath`.

Nach dem Einschalten (*Coasterbot setup*, beide Flags auf `False`) verharrt der Bot im zentralen Wartezustand *Coasterbot wartend* (LED aus), bis der Taster gedrückt wird. Welcher Folgezustand dann angesteuert wird, hängt von beiden Flags ab: ist der Tisch noch nicht eingelernt, folgt *Tisch einlernen*. Falls er eingelernt und kein Untersetzer ausgegeben wird, folgt *Untersetzer ausgeben*. Ist bereits einer ausgegeben, folgt stattdessen *Untersetzer einsammeln*. Jeder dieser drei Zustände kehrt nach Abschluss seiner Aktion automatisch in den Wartezustand zurück. Erkennt die Sicherheitskomponente währenddessen eine Tischkante oder scheitert die Pfadplanung (`bEdgedetected` bzw. `bNoPath`), wird sofort in *Not-Halt aktivieren* gewechselt (Bot stoppt, LED rot, Tischeinlernung verworfen). Ein erneuter Druck auf den Taster führt zurück in den Wartezustand. `Tisch einlernen` und

`Untersetzer ausgeben/einsammeln` entsprechen bereits den Methoden der Simulation `runLearnTable()` / `isTableLearned()` bzw. `runSpendCoaster()` / `runGetCoaster()` der neuen Klasse `CoasterbotFunctions` (`coasterbot_functions.{h,cpp}`, Betriebsart `MODE_COASTERBOT_LEARN_TABLE`). Die übrigen Zustände dieses Aufbaus (insbesondere *Not-Halt aktivieren*) sind darin noch nicht umgesetzt.

13 Simulationsmodell

Das Simulationsmodell erfüllt `NFA-SIM-001..005` und `SIM-001..007`: Die Robotersoftware läuft unverändert, die hardwarenahe Schicht wird durch Webots-Modelle ersetzt (`NFA-SIM-004`).

13.1 Robotermodell (PROTO)

`protos/coasterbot/Coasterbot.proto` ist aus der CAD-Baugruppe abgeleitet.

- **Koordinaten:** PROTO ist Y-up (lokal $+X$ rechts, $+Y$ oben, $-Z$ vorne). Die Welten sind Z-up (ENU). Instanzen stehen mit `rotation 1 0 0 1.5708` aufrecht.
- **Antrieb:** vier mal `RotationalMotor`, je Seite als coupled motors gekoppelt (`wheel_left::front` mit `wheel_left::rear`, analog rechts).
- **Sensoren:** vier mal `PositionSensor` an den Radgelenken (Encoder), `ultrasonic` (`DistanceSensor`, sonar, `lookupTable` in Metern, **analoges** Distanzsignal), vier `edge_*` (`DistanceSensor`, infrarot, nach unten, **digitales** Signal: `lookupTable` 0 0 / 0,098 0 / 0,10 1 / 4 1, eine reine 0/1-Stufe mit 2 mm numerischer Übergangszone statt einer analogen Rampe, siehe Abschnitt 4.2), `inertial_unit`, `gyro` und `accelerometer` (beide modellieren den laut Stückliste verbauten MPU-6050-Chip: `lookupTable` begrenzt auf dessen Werksdefault-Messbereich, `resolution` auf dessen 16-Bit-ADC-Schrittweite. Bias und Rauschen des Gyros kommen zusätzlich aus `WebotsHAL`, siehe Abschnitt 4.3).
- **Kollisionskörper:** zwei gestapelte Boxen (schmaler unterer Teil zwischen den Rädern) gegen Jittern.

Die `lookupTable`s der Distanzsensoren wurden so kalibriert, dass die Software mit demselben Signaltyp wie auf der Zielhardware arbeitet: Ultraschall liefert weiterhin ein analoges Distanzsignal in Metern (Vergleich mit `OBSTACLE_THRESHOLD_M`). Die Kantensensoren liefern ein digitales 0/1-Signal, das `WebotsHAL` nur noch auf seine Polarität hin abfragt (`EDGE_ACTIVE_HIGH`, siehe Abschnitt 4.2). Ein analoger Schwellenwert befindet sich nicht mehr im Code.

13.2 Welten

Tabelle 15: Simulationswelten mit Bezug zur Software.

Welt	Inhalt / zugehörige Betriebsart
`coasterbot.wbt`	Tisch mit Flaschen, `MODE_AUTONOMOUS`
`coasterbot-testfield.wbt`	offene 4 × 4-Fläche, `MODE_ODOMETRY_TEST`, `MODE_MOTOR_CONTROL_TEST`, `MODE_FAULT_INJECTION`
`coasterbot-astar.wbt`	zwei versetzte Wände (Slalom), Zielmarker, `MODE_NAVIGATE`
`coasterbot-dstar.wbt`	Wand mit zwei Lücken, unkartierte Box, `MODE_NAVIGATE_DSTAR`
`coasterbot-simtest.wbt`	`Table` 1,6 × 2,0, `FlowerPot` + `BeerBottle`s, `MODE_SIM_TEST`

Die Konstanten des Szenarios in `main.cpp` (Namensräume `nav_scenario`, `dstar_scenario`, `sim_scenario`) müssen mit der jeweiligen Welt übereinstimmen (Start, Ziel, Hindernispositionen, Tischgrenzen).

13.3 Reproduzierbarkeit

`SIM-001` / `ST-SIM-004` werden erfüllt, weil die Komponenten deterministisch sind: fester Zeitschritt, sortierte Prioritätswarteschlangen mit eindeutigem Schlüssel, und Zufallszahlen nur an zwei klar abgegrenzten Stellen: `RobotLogic` nutzt `std::rand` (an der Navigation nicht beteiligt), und die simulierte Gyro-Kalibrierung in `WebotsHAL` nutzt einen `std::mt19937`-Generator mit **festem** statt zeitbasiertem Seed (Abschnitt 4.3). Damit erzeugt jeder Lauf dieselbe Rauschfolge. `WorldInfo.randomSeed` wird in allen Coasterbot-Welten ebenfalls fest gesetzt statt auf dem Default 0 („aktuelle Zeit als Seed“) belassen, da Webots selbst ansonsten keine deterministische physikalische Simulation garantiert. Zwei Läufe von `MODE_SIM_TEST` liefern weiterhin byte-identische Protokolle.

14 Systemüberwachung

Die Systemüberwachung (`MON-001..005`) ist in der Simulation als strukturiertes Konsolenprotokoll umgesetzt. Jede Betriebsart gibt aus:

- **Betriebszustand** (`MON-003`): Zustandswechsel von `RobotLogic` (`[AUTONOMOUS] ...Zustand: ...`) und `SafetyMonitor` (`stateName()`), Pose alle 2 s bis 3 s.
- **Ereignisse**: Kantenerkennung, Erkennung eines unkartierten Hindernisses, Umplanung, Zielerreichung.
- **Fehlererkennung** (`MON-001`): „KEIN PFAD“, Zeitüberschreitung der Navigation, Guardabbruch (`guard hit`) in `DStarLite::computeShortestPath`.

- **Aufgabenergebnis** (`MON-004/005`): in `MODE_SIM_TEST` pro Testfall `BESTANDEN` bzw. `FEHLGESCHLAGEN` mit Kennzahlen (Restfehler zum Ziel, minimaler Hindernisabstand, Kantenereignisse, „Tisch verlassen“).

Nicht umgesetzt sind eine persistente Protokolldatei mit Zeitstempeln und die im Pflichtenheft skizzierte zentrale Ereignisverwaltung, das Konsolenprotokoll deckt den Prototyp-Bedarf ab.

15 Abbildung auf Anforderungen und Testfälle

Tabelle 16: Rückverfolgbarkeit Anforderung → Softwarekomponente → Test.

ID	Umsetzung	Nachweis
NAV-001	`DwaPlanner` (Ultraschall als virtuelles Hindernis), `OccupancyGrid`	ST-SIM-002
NAV-002	`DwaPlanner`, `DStarLite`	ST-SIM-002
NAV-003	`SafetyMonitor` (Kantensensoren)	ST-SIM-003
NAV-004	`SafetyMonitor`; Gitter auf Tischfläche begrenzt	ST-SIM-003
NAV-005	`DStarLite` + Kartenpflege in `main.cpp`	ST-SIM-002
NAV-006	`PoseEstimator` (Odometrie)	ST-SIM-001
NAV-007	`PoseEstimator` (Gyro-Fusion)	ST-SIM-001
NAV-008	`astarPlan` / `DStarLite` + `DwaPlanner`	ST-SIM-001
NAV-009	gesamter Navigationsstapel	ST-SIM-001/002
NAV-010	`OccupancyGrid`-Inflation, DWA-`clearance`, `SafetyMonitor`	ST-SIM-002 (min. Abstand)
NAV-011	*nicht umgesetzt* (keine Getränkeerkennung)	keine
SAF-001	`SafetyMonitor` (sofortiger `drive(0)`)	ST-SIM-003
SAF-002	`FaultMonitor` (Ultraschall-Plausibilitätsprüfung)	ST-SIM-005
SAF-003	`DwaPlanner` (Kollisions-/Bremsprüfung), `SafetyMonitor`	ST-SIM-002/003
SAF-004	`DwaPlanner`: `MAX_V` + Bremsbedingung	ST-SIM-002
SAF-005	`SafetyMonitor`-Übernahme; Taster über HAL-Ereignis	keine
MON-001	Konsolenprotokoll, „KEIN PFAD“, Wächter, Zeitüberschreitung	ST-SIM-*

ID	Umsetzung	Nachweis
MON-002	`stdout`-Protokoll (keine Datei)	ST-SIM-*
MON-003	`getStateName()` / `stateName()`, Pose-Ausgabe	ST-SIM-*
MON-004	`DwaPlanner::reachedGoal()`, Verdikt `p1`	ST-SIM-001
MON-005	`FEHLGESCHLAGEN`-Ausgabe, Zeitüberschreitung	ST-SIM-*
NFA-ARC-001/002	je Komponente eine Klasse/Datei, klare Trennung	Inspektion
NFA-ARC-003	HAL + entkoppelte Planer; neue Komponenten ohne Kernänderung	Inspektion
NFA-ARC-004	`RobotHAL`, `setPath`, `update`, `clearance`	Inspektion
NFA-ARC-005	`RobotHAL` / `WebotsHAL`	Inspektion
NFA-MAI-001/002	nur `webots_hal.cpp` ist hardwareabhängig	Inspektion
NFA-MAI-004	Komponenten nehmen nur `RobotHAL` / `OccupancyGrid`	Inspektion
NFA-SIM-001..005	Webots-Modell, `WebotsHAL`, identische Schnittstellen	ST-SIM-*
SIM-001	deterministische Komponenten	ST-SIM-004
SIM-004	`OccupancyGrid` + Welt-Hindernisse	ST-SIM-002
SIM-005	`WebotsHAL` (`injectUltrasonicFault`/`injectLeftMotorFault`)	ST-SIM-005/006

15.1 Status der Simulationstestfälle

Testfall	Gegenstand	Status
ST-SIM-001	Navigation zu einem Zielpunkt	umgesetzt, `MODE_SIM_TEST` Phase 1
ST-SIM-002	Dynamische Hindernisvermeidung	umgesetzt (bekannte + unkartierte Hindernisse)
ST-SIM-003	Verhindern des Tischverlassens	umgesetzt, `MODE_SIM_TEST` Phase 2
ST-SIM-004	Wiederholbarkeit	umgesetzt (zwei Läufe, byteidentisch)
ST-SIM-005	Sensorausfall	umgesetzt, `MODE_FAULT_INJECTION` Phase 1 (erkannt $< 0.02\,\mathrm{s}$)

Testfall	Gegenstand	Status
ST-SIM-006	Aktorfehler	umgesetzt, `MODE_FAULT_INJECTION` Phase 2 (erkannt nach 0.34 s)
ST-SIM-007	Untersetzerhandling	außerhalb des Umfangs (keine Manipulation)
ST-SIM-008	Validierung Simulation vs. Prototyp	offen (Prototyp erforderlich)

16 Grenzen und offene Punkte

Einige Anforderungen sind bisher nicht Teil der in diesem Dokument beschriebenen Simulationssoftware (`controllers/coasterbot/`). Die missionsbezogene Ablaufsteuerung `IDLE` → … → `DONE` ist darin noch nicht vollständig implementiert. Der Ablauf ist in Abschnitt 12.2 dargestellt. Das Untersetzerhandling (Aufnahme/Ablage, `CST-*`) wird in einem eigenen Stand in der Simulation (`coasterbot_V_01_00`, Klasse `CoasterbotFunctions`) über einen Zustandsautomaten für Ausgabe und Aufnahme umgesetzt. Eine Manipulationsplanung über IK + RRT* ist dafür nicht erforderlich, da die konstruierte Mechanik (Hubscheibe mit Ausschieber) nur einen einzigen, servogesteuerten Freiheitsgrad besitzt. Die Lokalisierung nutzt statt EKF/UKF einen Komplementärfilter, der für die ebene Tischfläche ausreicht, jedoch keine Schätzung der Unsicherheit liefert. Der reale Aufbau des Roboters verfügt inzwischen, anders als zum Auslegungszeitpunkt des Simulationsmodells angenommen, über einen Encoder je Seite (siehe Abschnitt 4.2). Eine Gegenüberstellung der damit erreichten realen Odometriegenauigkeit mit dem Simulationsmodell steht noch aus. Der Ultraschall an der Front findet kleine, außeraxiale Tischobjekte nicht zuverlässig. Deshalb sind im Simulationstest Blumentopf und Flaschen als bekannte Hindernisse in der Karte hinterlegt, während die reine Sensor-Entdeckung separat mit einem großen Hindernis in `MODE_NAVIGATE_DSTAR` gezeigt wird. Die zwei getrennten Gitter für Planung und DWA samt `centerPath` könnten verbessert werden. Eine gradientenbasierte Kostenkarte würde beide zu einer Einzigen zusammenführen. `FaultMonitor` (Abschnitt 10.3) deckt den in `ST-SIM-005/006` geforderten Fall ab (Ultraschall- bzw. Ausfall des linken Motors). Offen bleiben außerdem die Validierung gegen den realen Prototyp (`ST-SIM-008`) sowie die Eignung von `getGyroZ()` für eine reale IMU, deren Dater lediglich aus dem Datenblatt entnommen wurden und bislang nicht abschließend beurteilt wurde.

17 Implementierung auf Hardware

Beim Umsetzen des Hardware-Abstraktions-Layers(HAL) der Simulation auf der realen Raspberry Pi Pico 2 Microcontroller Architektur sind einige Besonderheiten implementiert, die hier beschrieben werden sollen.

17.1 Non-blocking Implementierung

Viele Funktionalitäten sind übertragen auf eine Programmiersprache ein einfacher Befehl, der eine Absicht darstellt, was mit dem System passieren soll. Ein Beispiel soll dazu der Befehl „Anhalten“ für die Motorsteuerung sein. Ein solcher Befehl muss selbstverständlich auf die Hardware umgelegt werden, er beeinhaltet aber auch ein zeit- sowie zustandsvariantes Verhalten der Komponenten:

- Wenn sich das System derzeit nicht bewegt, muss nichts weiter unternommen werden
- Wenn sich das System derzeit vorwärts bewegt müssen die Motoren zunächst auf Bremsen gesetzt werden, dieser Zustand muss für eine gewisse Zeit gehalten werden, möglicherweise abhängig von der vorherigen Geschwindigkeit, anschließend kann die Motorsteuerung in einen Idle Zustand gesetzt werden.

Dieses Verhalten kann selbstverständlich sequentiell mit einer Wartezeit implementiert werden, im Arduino Befehlssatz wäre dies ein `delay(unsigned milliseconds);` Befehl. Ein solches Delay hat aber den Nachteil, dass alle weiteren Aufgaben während dieser Wartezeit ebenfalls unterbrochen werden und die Ansteuerung nicht responsiv ist.

Eine Lösung für dieses Verhalten, die im vorliegenden Projekt so implementiert wurde, ist eine State-Machine mit automatischen und zeitabhängigen Zustandswechseln. In der Hauptroutine wird nach dem Auswerten aller Sensordaten für die Motorsteuerung lediglich die Absicht gesetzt, z.B. `motor_control.stop();`. Die Methode Stop setzt dann wiederum nur einen internen Zustandsautomaten auf einen `Stop` Zustand. Unabhängig von der gesetzten Absicht wird mit jeder Iteration der Hauptroutine eine `motor_control.update();` Methode aus der Hauptroutine aufgerufen, in der die eigentlichen Änderungen vorgenommen werden. Hier wird dem Zustand entsprechend weiter vorgegangen:

- Wenn `Zustand == Stop`, dann setze Motoren auf Bremsen, speichere den aktuellen Zeitstempel in `_stop_begin` und setze Zustand auf `Stopping`.
- Wenn `Zustand == Stopping`, dann prüfe ob aktueller Zeitstempel größer als `_stop_begin + _stop_duration` ist. Falls nein return. Falls ja, dann setze Motoren auf Idle und setze Zustand auf `Idle`.

Durch eine solche interne Logik bedarf es keiner Delays, alle Komponenten bleiben responsiv und es wird dennoch nur wirklich mit der Hardware interagiert, wenn es notwendig ist. In der Praxis wird dies etwas komplexer durch die folgenden gespeicherten Zustände implementiert:

```
enum Motion { STOPPED, FORWARD, BACKWARD, TURNINGLEFT, TURNINGRIGHT }; // Possible
    device motion
  enum Phase { // current state
    UNINITIALIZED, // begin() has not run yet, update() keeps its hands off the pins
    IDLE, // at rest, driver on standby, no deadline
```

```
  BRAKING, // both sides shorted with the driver still enabled, until _deadline
  COASTING, // driver released, dead time after braking, until _deadline
  ARMING, // direction pins already set, driver still released, until _deadline
  RUNNING // driver enabled, the wheels do what was requested
};

Phase _phase {UNINITIALIZED};
Motion _requested {STOPPED}; // what the main loop asked for
Motion _applied {STOPPED}; // what is currently wired to the driver inputs
Motion _driven {STOPPED}; // what was last engaged, still valid while braking and
    coasting down
```

Die Zustände sind also geringfügig komplexer und es wird zwischen interner Statemachine und einer extern angefragten Absicht unterschieden. Zum Zwecke der Übersichtlichkeit wurde dies aber im obigen Beispiel etwas vereinfacht.

Das Prinzip des non-blocking Verhaltens ist in allen Software Komponenten umgesetzt. So ist es z.B. auch möglich während des Bremsvorgangs weiter Sensorwerte auszuwerten, wie z.B. den Inertialsensor, oder ein Blinken der LED zeitkorrekt aufrechtzuerhalten.

17.2 Inertialsystem Normierung

Das Inertialsystem wurde mit dem Sensor MPU-6050 implementiert. Dieser ist über I^2C angebunden und liefert Beschleunigungswerte für eine x, y und z Richtung, sowie drei Winkelbeschleunigungen für diese Achsen. Die rohen Werte werden dann als int16 big endian Werte geliefert, werden dann mit $1/8192$ normiert und sind dann als Vielfache der Erdbeschleunigung $g = 9.81 m/s^2$ zu interpretieren. Problematisch bei diesen Werten ist allerdings, dass die rohen Sensorwerte mit einem Mittelwert ungleich 0 behaftet sind(Bias), sowie die Lage des Sensors nicht eineindeutig im Raum ist(Orientierung). Dies ist normales Grundverhalten des Sensors und muss softwareseitig ausgeglichen werden.

Es bedarf also bei Systemstart sowie im gebremsten Idle Zustand einer fortlaufenden Messung und Berechnung des Bias Wertes auf Basis der rohen Messwerte. Dieser Bias Wert muss also von jedem Messwert anschließend zunächst abgezogen werden. Praktisch erfolgt dies bei Initialisierung über fortlaufende Messungen für 500 Millisekunden, in denen die Messungen akkumuliert und anschließend durch die Anzahl der Messungen geteilt werden. Dieser Roh-Wert wird im laufenden Betrieb dann immer von den aktuellen Messungen subtrahiert. Wenn die Motorsteuerung im Zustand `Idle` ist wird wiederum ein Flag des Inertialsystems gesetzt, wodurch die nächste Messung, falls der Sensor tatsächlich für einige Zeit nicht weiter beschleunigt wurde, mit einem Tiefpass Filter auf den aktuellen Bias Wert zugerechnet wird:

```
constexpr float bias_alpha = 0.005f;
_bias_x += bias_alpha * (x_value - _bias_x);
_bias_y += bias_alpha * (y_value - _bias_y);
_bias_z += bias_alpha * (z_value - _bias_z);
```

Die Orientierung des Sensors ist auf der Hardwareanordnung niemals perfekt symmetrisch, dies muss in der Kalibrierungsphase zu Beginn vermessen werden. Dazu wird die Rodrigues-Formel[1] genutzt und der maximale Sensorwert in Richtung der z-Achse normiert, was der Erdbeschleunigung entspricht.

Die somit normalisierten und reorientierten Beschleunigungswerte können nun zur Inertialnavigation genutzt werden. Dazu werden die Beschleunigungswerte bei Vorwärts- oder Rückwärtsfahrt mit linearer Approximation zu Geschwindigkeitswerten integriert, aus welchen sich wiederum die aktuelle Position ergibt. Die bei der Integration entstehenden Fehler sind nicht unerheblich und führen zu einer Unsicherheit auf den Positionsdaten, innerhalb der biherigen Hardwaretests ergab sich der Fehler aber als akzeptabel.

18 Anhang A: Schnittstelle `RobotHAL`

```
enum WheelId { WHEEL_LEFT = 0, WHEEL_RIGHT = 1 }; // ein Encoder je Seite

class RobotHAL {
public:
    virtual ~RobotHAL() {}

    virtual bool step() = 0; // ein Regelzyklus

    virtual void setLeftSpeed(float radPerSec) = 0; // linke Seite (gekoppelt)
    virtual void setRightSpeed(float radPerSec) = 0; // rechte Seite (gekoppelt)

    virtual float getUltrasonicDistance() = 0; // [m]
    virtual bool getEdgeFrontLeft() = 0; // true = Kante, kalibriert in der HAL
    virtual bool getEdgeFrontRight() = 0;
    virtual bool getEdgeRearLeft() = 0;
    virtual bool getEdgeRearRight() = 0;

    virtual float getYaw() = 0; // [rad]
    virtual float getGyroZ() = 0; // [rad/s]
    virtual float getWheelAngle(WheelId wheel) = 0; // [rad] EINER SEITE, + =
        vorwaerts
    virtual float getTime() = 0; // [s]
};
```

19 Anhang B: Wesentliche Parameter

Komponente	Parameter	Wert
Geometrie	Radradius r	0.035 m
	Spurweite b	0.202 m

[1] https://de.wikipedia.org/wiki/Rodrigues-Formel

Komponente	Parameter	Wert
	Radstand	0.113 m
PoseEstimator	HEADING_ODO_WEIGHT	0,05
OccupancyGrid	Auflösung	0.05 m
	OCC_COMMIT_THRESHOLD (Abschnitt 6.3)	0,8
	DEFAULT_SMOOTHING	0,5
astar	centerPath maxShift	0.20 m
DwaPlanner	MAX_V / MAX_W	$0.18\,\mathrm{m\,s^{-1}}$ / $2.6\,\mathrm{rad\,s^{-1}}$
	ACC_V / ACC_W	$0.5\,\mathrm{m\,s^{-2}}$ / $6\,\mathrm{rad\,s^{-2}}$
	SIM_TIME / SIM_DT	1.5 s / 0.1 s
	Gewichte w_h, w_p, w_c, w_v	1,0 / 2,0 / 1,5 / 0,3
	CARROT_DIST / GOAL_TOL	0.30 m / 0.05 m
	ROBOT_RADIUS	0.14 m
WebotsHAL	EDGE_ACTIVE_HIGH (Kantensensor-Polarität)	wahr
	GYRO_BIAS_RAD (Abschnitt 4.3)	$0.0524\,\mathrm{rad\,s^{-1}}$ ($\triangleq 3\,^\circ\,\mathrm{s^{-1}}$)
	GYRO_NOISE_STD_RAD	$0.000\,873\,\mathrm{rad\,s^{-1}}$ ($\triangleq 0.05\,^\circ\,\mathrm{s^{-1}}$-rms)
	GYRO_CAL_SAMPLES	32 (≈ 0.5 s)
	MOTOR_GAIN_LEFT_TRUE/RIGHT_TRUE (Abschnitt 4.4)	0,95 / 1,05
	MOTOR_KI (Closed-Loop-Integralregler)	25
	MOTOR_MAX_OMEGA (Anti-Windup)	$20\,\mathrm{rad\,s^{-1}}$
SafetyMonitor	EVADE_RADS / EVADE_TIME	$3\,\mathrm{rad\,s^{-1}}$ / 0.7 s
	PAUSE_TIME	0.3 s
FaultMonitor	ULTRASONIC_MAX_VALID (Abschnitt 10.3)	4.5 m
	SPEED_DEVIATION_RADS / MIN_COMMANDED_RADS	$1\,\mathrm{rad\,s^{-1}}$ / $0.5\,\mathrm{rad\,s^{-1}}$
	FAULT_DEBOUNCE_TIME	0.3 s
RobotLogic	OBSTACLE_THRESHOLD_M	0.22 m
	CRUISE_SPEED / TURN_SPEED	$6\,\mathrm{rad\,s^{-1}}$ / $4\,\mathrm{rad\,s^{-1}}$